Assignment 1 - Analysis of Algorithms
Sorting Race Results with Two Algorithm

1. Objective and method

Implement and compare Merge Sort and stable LSD Radix Sort for non-negative integer finish times in the range 0–9,999,999. Built-in sorting is used only as an external correctness reference, not inside either algorithm.

Five reproducible random datasets were generated (n = 1,000; 5,000; 10,000; 50,000; 100,000) using Python's random.Random(20261025). Each algorithm was run three times per size on the same dataset. The reported time is the median of the three runs. Input copying, reference sorting, validation, data generation, and plotting were outside the timed section.

2. Correctness verification

Both algorithms passed all eight test cases: empty and single-element inputs, unsorted and duplicate values, already sorted and reverse-sorted inputs, boundary values, and mixed digit lengths. Each test checked ascending order, preservation of all values, and unchanged input. LSD Radix Sort also rejects negative values.

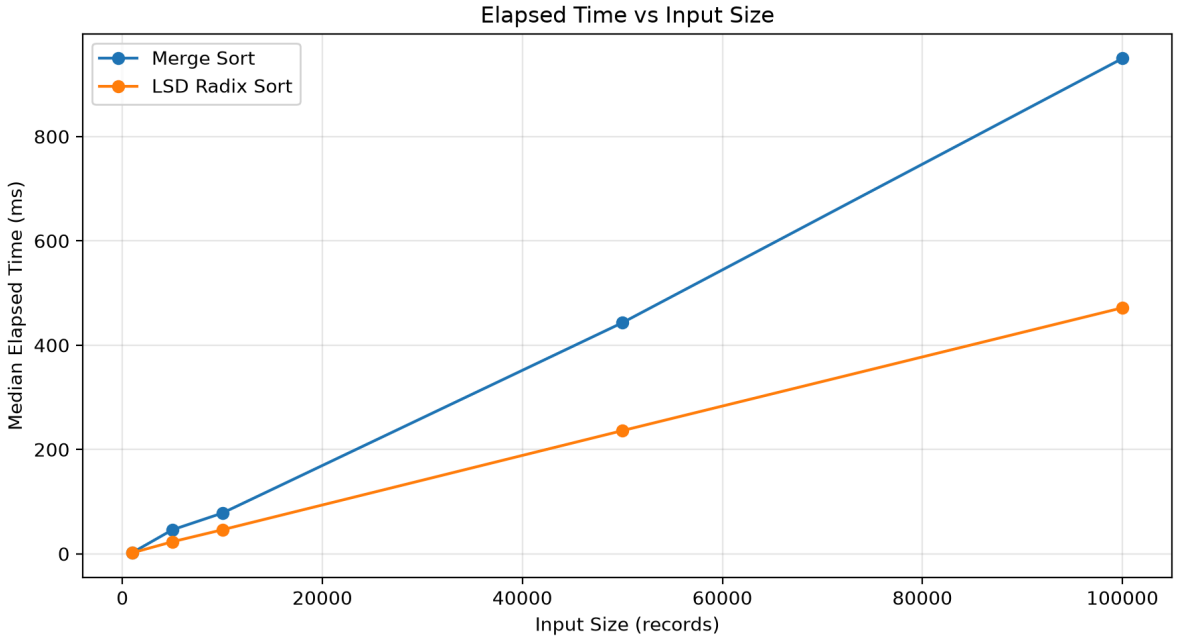
Algorithm	**Tests passed**	**Checks**
Merge Sort	8/8	Sorted output; same values; input unchanged
LSD Radix Sort	8/8	Sorted output; same values; input unchanged

3. Empirical results

Median elapsed time (ms); values are the measurements recorded in the notebook.

Input size	**Merge Sort (ms)**	**LSD Radix Sort (ms)**	**Merge/Radix**
1	1/2	1.9764	1.18×
5	46.0119	23.0984	1.99×
10	78.0762	46.0919	1.69×

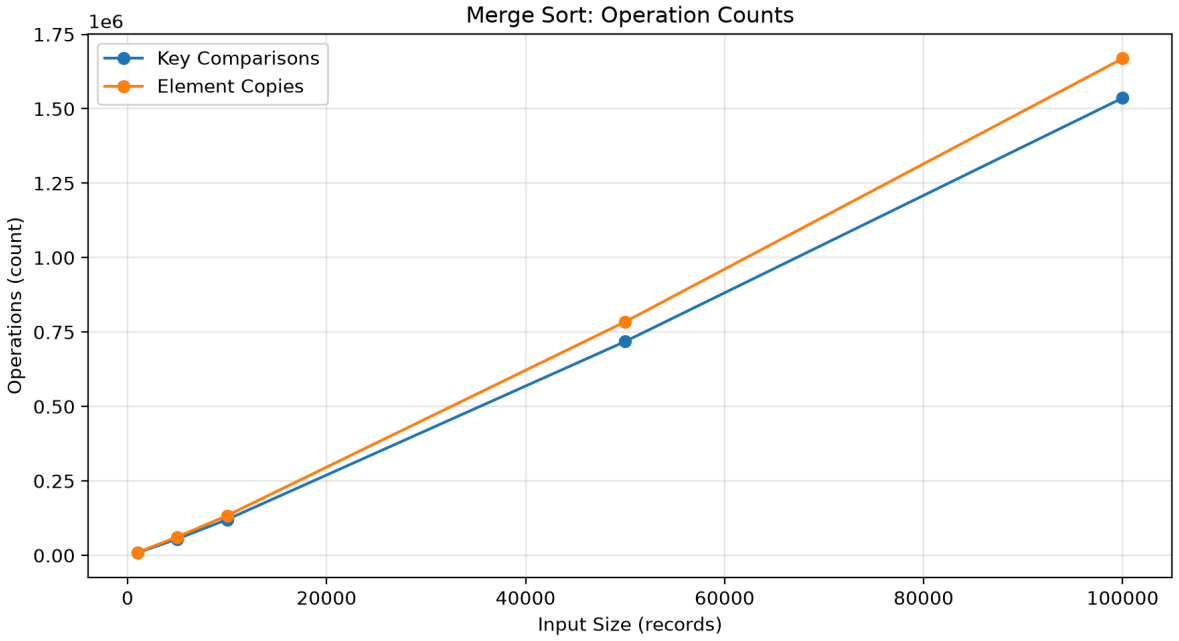
50	443.0405	236.1207	1.88×
100	949.6000	471.6367	2.01×



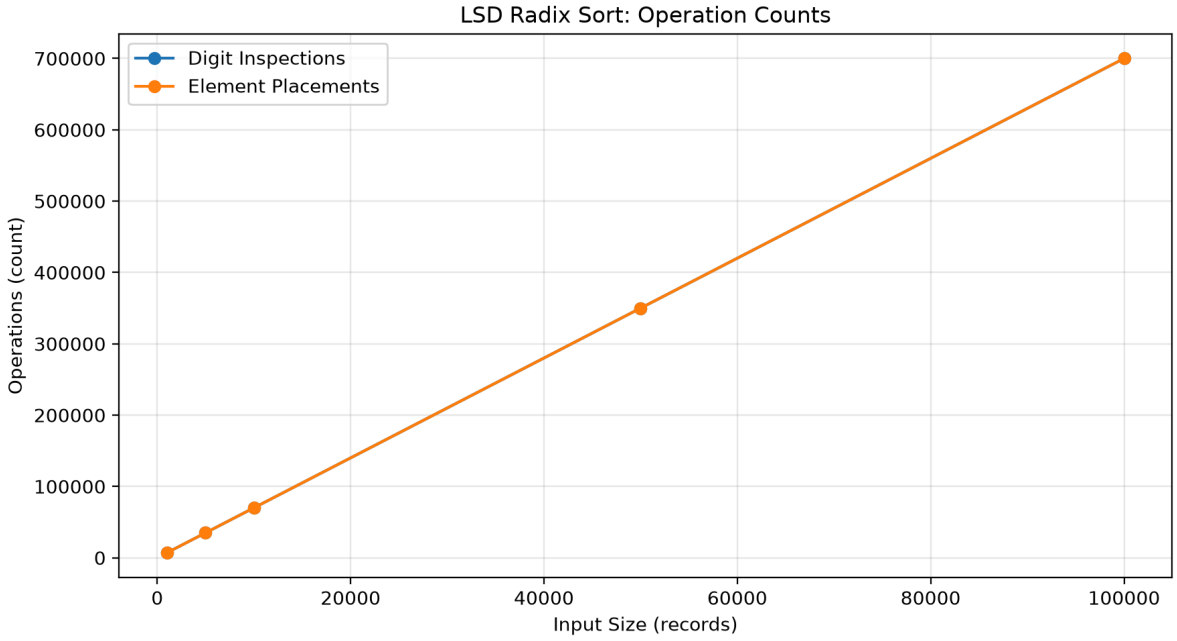
4. Operation counts

Counters are algorithm-specific and must not be compared directly across algorithms. Merge Sort counts element comparisons in merge loops and element appends to merged outputs; the counter does not include list slicing or base-case copies. Radix Sort counts digit inspections in the frequency-counting pass and output placements; digit extraction in the placement pass is not included in the inspection counter. Counts below are deterministic for the specified seed and counter definitions.

Input size	Comparisons	Element appends
1	8,698	9,976
5	55,24	61,808
10	120,36	133,616
50	718,271	784,464
100	1,536,235	1,668,928



Input size	Digit inspections	Placements
1	7	7
5	35	35
10	70	70
50	350	350
100	700	700



5. Complexity and interpretation

Algorithm	Best	Average	Worst	Auxiliary space
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
LSD Radix Sort	O(d(n + b))	O(d(n + b))	O(d(n + b))	O(n + b)

Here d is the number of decimal digits and b = 10 is the base. Because the input range is bounded at 9,999,999, d = 7 and b are constants, so Radix Sort has linear time with respect to n for this task. Merge Sort has O(n log n) time regardless of input order. The measured Radix Sort was faster at every tested size; at n = 100,000 it was about 2.01× faster. This matches the expected advantage of processing a fixed number of digits rather than repeatedly comparing values. Actual times are implementation- and machine-dependent.

6. Reflection

The experiment confirmed that both algorithms correctly sort the tested finish times, but their performance differs. LSD Radix Sort was faster for all dataset sizes, taking about half the time of Merge Sort at 100,000 values. This advantage is expected because the values have at most seven decimal digits, allowing Radix Sort to process a fixed number of passes. In contrast, Merge Sort has a time complexity of O(n log n). The operation counters measure different types of work: Merge Sort counts comparisons and element appends, while Radix Sort counts digit inspections and

placements. Therefore, these counts should not be compared directly. The results apply to non-negative integers within the specified range; a wider key range may reduce Radix Sort's advantage. Finally, execution times depend on the Python implementation and hardware, so the findings reflect this experiment rather than a universal ranking.

7. Source code

```
import random
import time
import statistics
import pandas as pd
import matplotlib.pyplot as plt

# Reproducible experiment settings
SEED = 20261025
MAX_VALUE = 9_999_999
DATASET_SIZES = [1_000, 5_000, 10_000, 50_000, 100_000]
NUM_TRIALS = 3

random.seed(SEED)

print(DATASET_SIZES)
print(SEED)

def merge_sort(arr):
    """
    Sort a list using Merge Sort.

    Returns:
        sorted_arr: sorted copy of the input list
        stats: operation counts
    """
    stats = {
        "comparisons": 0,
        "copies": 0
    }

    def sort(items):
        if len(items) <= 1:
            return items.copy()

        mid = len(items) // 2
        left = sort(items[:mid])
        right = sort(items[mid:])

        merged = []
```

```
        i = 0
        j = 0

        while i < len(left) and j < len(right):
            stats["comparisons"] += 1

            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1

            stats["copies"] += 1

        while i < len(left):
            merged.append(left[i])
            i += 1
            stats["copies"] += 1

        while j < len(right):
            merged.append(right[j])
            j += 1
            stats["copies"] += 1

        return merged

    return sort(arr), stats


test_cases = [
    [],
    [5],
    [3, 1, 2],
    [4, 2, 2, 4, 1],
    [1, 2, 3, 4, 5],
    [5, 4, 3, 2, 1],
    [10, 1, 100, 0, 9_999_999],
    [100, 10, 1, 1_000, 10]
]

for index, test_data in enumerate(test_cases, start=1):
    result, stats = merge_sort(test_data)
    assert result == sorted(test_data), f"Test {index} failed"
    assert test_data == test_data.copy(), f"Input changed in test {index}"
    print(f"Test {index}: PASS")

print(f"\nAll {len(test_cases)}")
```

```
def lsd_radix_sort(arr):
    """
    Stable LSD Radix Sort for non-negative integers.

    Returns:
        sorted_arr: sorted copy of the input list
        stats: operation counts
    """
    if any(value < 0 for value in arr):
        raise ValueError("LSD Radix Sort supports non-negative integers only.")

    result = arr.copy()

    stats = {
        "digit_inspections": 0,
        "placements": 0
    }

    if not result:
        return result, stats

    max_value = max(result)
    exp = 1
    base = 10

    # Process every decimal digit, including leading zero digits.
    while exp <= 10**6:
        output = [0] * len(result)
        count = [0] * base

        # Count occurrences of each digit.
        for value in result:
            digit = (value // exp) % base
            count[digit] += 1
            stats["digit_inspections"] += 1

        # Convert counts into cumulative positions.
        for i in range(1, base):
            count[i] += count[i - 1]

        # Traverse backwards to preserve stability.
        for i in range(len(result) - 1, -1, -1):
            value = result[i]
            digit = (value // exp) % base
            count[digit] -= 1
            output[count[digit]] = value
```

```
            stats["placements"] += 1

        result = output
        exp *= base

    return result, stats


for index, test_data in enumerate(test_cases, start=1):
    result, stats = lsd_radix_sort(test_data)
    assert result == sorted(test_data), f"Test {index} failed"
    assert test_data == test_data.copy(), f"Input changed in test {index}"
    print(f"Test {index}: PASS")

print(f"\nAll {len(test_cases)}")

# Verify that negative numbers are rejected.
try:
    lsd_radix_sort([3, -1, 2])
    raise AssertionError("Negative-number test failed")
except ValueError:
    print("Negative-number validation: PASS")

def generate_datasets():
    """Generate reproducible datasets for all experiment sizes."""
    rng = random.Random(SEED)
    datasets = {}

    for size in DATASET_SIZES:
        datasets[size] = [
            rng.randint(0, MAX_VALUE)
            for _ in range(size)
        ]

    return datasets


datasets = generate_datasets()

for size, data in datasets.items():
    print(
        f"n={size:>6}: "
        f"min={min(data):>7}, "
        f"max={max(data):>7}, "
        f"records={len(data)}"
    )
```

```
def benchmark_algorithm(sort_function, data, expected):
    """
    Measure sorting time over repeated runs.
    Data copying and correctness checks are excluded from timing.
    """
    elapsed_times = []
    operation_stats = None

    for _ in range(3):
        input_data = data.copy()

        start = time.perf_counter()
        result, stats = sort_function(input_data)
        end = time.perf_counter()

        assert result == expected, "Sorting result is incorrect!"

        elapsed_times.append(end - start)
        operation_stats = stats

    median_ms = statistics.median(elapsed_times) * 1000

    return median_ms, operation_stats


results = []

for size, data in datasets.items():
    expected = sorted(data)

    for algorithm_name, sort_function in [
        ("Merge Sort", merge_sort),
        ("LSD Radix Sort", lsd_radix_sort)
    ]:
        median_ms, stats = benchmark_algorithm(
            sort_function,
            data,
            expected
        )

        row = {
            "Input Size": size,
            "Algorithm": algorithm_name,
            "Median Time (ms)": round(median_ms, 4)
        }
        row.update(stats)
        results.append(row)
```

```
        print(
            f"n={size:>6} | "
            f"{algorithm_name:<14} | "
            f"{median_ms:>10.4f} ms"
        )

results_df = pd.DataFrame(results)

display(results_df)

# Table comparing execution times
time_table = results_df.pivot(
    index="Input Size",
    columns="Algorithm",
    values="Median Time (ms)"
).reset_index()

display(time_table)

# Separate operation-count tables
merge_table = results_df[
    results_df["Algorithm"] == "Merge Sort"
].drop(columns=["Algorithm", "Median Time (ms)"]).reset_index(drop=True)

radix_table = results_df[
    results_df["Algorithm"] == "LSD Radix Sort"
].drop(columns=["Algorithm", "Median Time (ms)"]).reset_index(drop=True)

print("Merge Sort operation counts:")
display(merge_table)

print("LSD Radix Sort operation counts:")
display(radix_table)


# Chart 1: Execution time comparison
plt.figure(figsize=(9, 5))

plt.plot(
    time_table["Input Size"],
    time_table["Merge Sort"],
    marker="o",
    label="Merge Sort"
)

plt.plot(
    time_table["Input Size"],
    time_table["LSD Radix Sort"],
```

```
    marker="o",
    label="LSD Radix Sort"
)

plt.title("Elapsed Time vs Input Size")
plt.xlabel("Input Size (records)")
plt.ylabel("Median Elapsed Time (ms)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("elapsed_time_comparison.png", dpi=200)
plt.show()


# Chart 2: Merge Sort operation counts
plt.figure(figsize=(9, 5))

plt.plot(
    merge_table["Input Size"],
    merge_table["comparisons"],
    marker="o",
    label="Key Comparisons"
)

plt.plot(
    merge_table["Input Size"],
    merge_table["copies"],
    marker="o",
    label="Element Copies"
)

plt.title("Merge Sort: Operation Counts")
plt.xlabel("Input Size (records)")
plt.ylabel("Operations (count)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("merge_sort_operations.png", dpi=200)
plt.show()


# Chart 3: LSD Radix Sort operation counts
plt.figure(figsize=(9, 5))

plt.plot(
    radix_table["Input Size"],
    radix_table["digit_inspections"],
    marker="o",
```

```
    label="Digit Inspections"
)

plt.plot(
    radix_table["Input Size"],
    radix_table["placements"],
    marker="o",
    label="Element Placements"
)

plt.title("LSD Radix Sort: Operation Counts")
plt.xlabel("Input Size (records)")
plt.ylabel("Operations (count)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("radix_sort_operations.png", dpi=200)
plt.show()


# Save detailed benchmark results
results_df.to_csv("sorting_results.csv", index=False)

# Save the execution-time comparison table
time_table.to_csv("sorting_time_summary.csv", index=False)

# Save operation-count tables
merge_table.to_csv("merge_sort_operations.csv", index=False)
radix_table.to_csv("radix_sort_operations.csv", index=False)

print("Results exported successfully:")
print("- sorting_results.csv")
print("- sorting_time_summary.csv")
print("- merge_sort_operations.csv")
print("- radix_sort_operations.csv")
print("- elapsed_time_comparison.png")
print("- merge_sort_operations.png")
print("- radix_sort_operations.png")


summary = time_table.copy()

summary["Speedup (Merge / Radix)"] = (
    summary["Merge Sort"] / summary["LSD Radix Sort"]
).round(2)

display(summary)
```

```
print("\nFinal comparison at n = 100,000:")
last = summary.iloc[-1]
print(f"Merge Sort: {last['Merge Sort']:.4f} ms")
print(f"LSD Radix Sort: {last['LSD Radix Sort']:.4f} ms")
print(f"Speedup: {last['Speedup (Merge / Radix)']:.2f}x")


def verify_algorithm(sort_function, test_data):
    original = test_data.copy()
    result, stats = sort_function(test_data)

    # Check that the output is in ascending order.
    is_sorted = all(
        result[i] <= result[i + 1]
        for i in range(len(result) - 1)
    )

    # Check that no values were lost, added, or changed.
    preserves_values = (
        len(result) == len(original)
        and sorted(result) == sorted(original)
    )

    # Check that the original input remains unchanged.
    input_preserved = test_data == original

    assert is_sorted, "Output is not sorted."
    assert preserves_values, "Input values were not preserved."
    assert input_preserved, "Original input was modified."

    return {
        "sorted": is_sorted,
        "same_values": preserves_values,
        "input_unchanged": input_preserved
    }


for algorithm_name, algorithm in [
    ("Merge Sort", merge_sort),
    ("LSD Radix Sort", lsd_radix_sort)
]:
    print(f"\n{algorithm_name}")

    for i, test_data in enumerate(test_cases, start=1):
        checks = verify_algorithm(algorithm, test_data)
        print(f"Test {i}: PASS — {checks}")
```